



Vishnu Waman Thakur Charitable Trust's
VIVA Institute of Technology

(Approved by AICTE, New Delhi, DTE, Govt. of Maharashtra and Affiliated to the University of Mumbai)
Shirgaon, Kumbharpada, Virar (E), Taluka-Vasai-401305, Phone no: 7770002544 Website: www.viva-technology.org

ACCREDITED by NAAC with "B++" Grade

DEPARTMENT OF COMPUTER ENGINEERING

Traffic Signal Automation Simulator

Group Members

Kush Atmaram Sutar
Rohit Dnyaneshwar Karwande
Sarthak Namdeo Patil

Guide Name: Prof. Saniket Kudoo

Traffic Signal Automation Simulator

Traditional traffic signals use fixed timers, leading to inefficient traffic flow, unnecessary congestion, and increased pollution. This project develops an interactive, web-based simulation of an adaptive traffic signal that dynamically allocates green light durations based on real-time traffic volume.



Introduction

The Problem of Congestion

The Bottleneck

Urban traffic congestion is a major global issue, resulting in wasted time, economic loss, and significant environmental impact.

A primary cause is the inefficiency of traditional, fixed-time traffic signals. These systems cannot adapt to fluctuating traffic demands throughout the day.

The "Smart" Solution

Adaptive Traffic Control Systems (ATCS) offer a solution. These systems use real-time data to adjust signal timings dynamically, prioritizing heavier traffic flow.

Project Aim: To build an accessible, web-based simulation to model, test, and visualize the core logic of an ATCS.

Review of Literature



Fixed-Time Controllers

The most basic form of traffic control. Timings are pre-calculated based on historical traffic data. They are simple but highly inefficient during off-peak or unexpected traffic patterns.



Actuated Controllers

Use sensors (like inductive loops) to detect the presence of vehicles. They can extend a green light for an arriving car but are not fully "adaptive" to overall volume.



Adaptive Control Systems

Real-world, large-scale systems (e.g., SCATS, SCOOT) that use a network of sensors to collect data on traffic volume and density. They adjust signal timings across multiple intersections to optimize traffic flow for an entire area.



Computer Vision & AI

The latest trend involves using CCTV cameras and AI-powered object detection (like YOLO) to count vehicles in real-time, providing a rich data source for adaptive systems.

Research Gap

Complexity of Real Systems

Most real-world adaptive systems are complex, proprietary, and expensive "black boxes," making their logic difficult to study or demonstrate.

Lack of Visualization Tools

There is a gap in simple, accessible, and interactive tools for educational and demonstration purposes. It's one thing to read about adaptive logic; it's another to see it in action.

Project's Contribution

This project fills that gap by providing a clear, open-source, and browser-based simulation. It makes the core principles of adaptive control easy to understand for students, stakeholders, and the public.

Problem Definition

Problem: To design a system that overcomes the inefficiencies of fixed-time traffic signals.

Objective: To develop a web-based, visual simulation of a four-way intersection with a smart, adaptive traffic controller.

Core Requirements:

01

The system must simulate four directions (North, South, East, West).

02

Traffic volume for each direction must be adjustable (simulating real-time data).

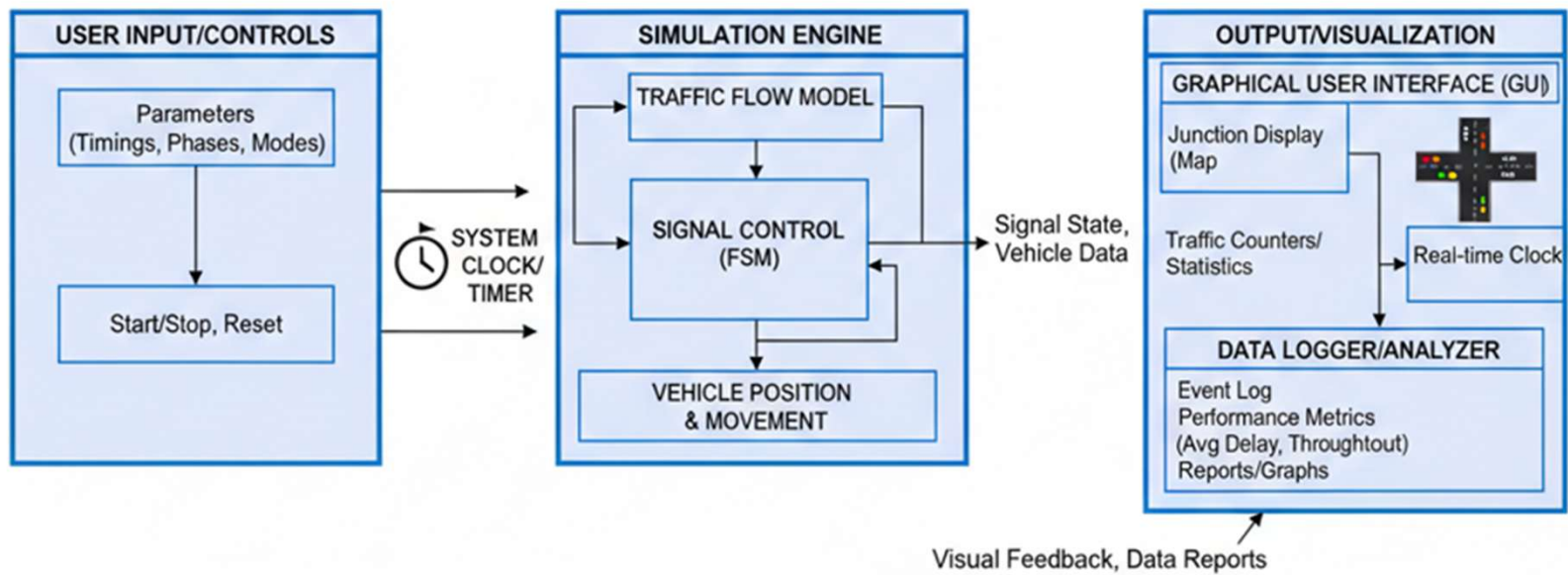
03

The controller logic must automatically calculate and assign an appropriate green light duration (from a set of realistic time slots) to the direction with the heaviest traffic.

04

The simulation must be dynamic, with traffic numbers changing to reflect cars passing (green) and cars arriving (red).

Block Diagram



Methodology

1 System Architecture

A client-side, single-page application.

2 State Management

A central JavaScript object `trafficVolumes` acts as the single source of truth for the entire simulation.

3 Control Logic

The system operates in a continuous, two-phase loop: (1) North-South cycle, (2) East-West cycle.

At the start of each cycle, the average traffic for that phase is calculated (e.g., $(\text{north} + \text{south}) / 2$).

A function `getGreenTimeForTraffic()` maps this average to a predefined time slot: 20s, 30s, 45s, or 60s.

4 Dynamic Simulation

A `setInterval` function runs every 500ms to update traffic volumes. It subtracts from green light directions and adds to red light directions.

5 Asynchronous Timing

JavaScript's `setTimeout` and `setInterval` are used to manage the precise sequencing of light changes (Green -> Yellow -> Red) and to display the live countdown.

Hardware/Software Requirements

Hardware Requirements (Client-Side)

- **CPU:** Any modern dual-core processor (e.g., Intel Core i3, AMD Ryzen 3).
- **RAM:** 2 GB or more.
- **Device:** Any standard PC, laptop, or smart device.
- **Input:** Keyboard and mouse, or touchscreen.

Software Requirements

- **Operating System:** Any (Windows, macOS, Linux, Android, iOS).
- **Runtime Environment:** A modern web browser (e.g., Google Chrome, Firefox).
- **Front-End Stack:** HTML5 (for structure), CSS3 (for styling), JavaScript (ES6+) (for all logic)
- **Libraries/Frameworks:** Tailwind CSS (for rapid UI development), Google Fonts (for typography)

Partial Results

Successful Implementation: A fully functional and interactive web-based simulator was developed.

1

Scenario 1: Balanced Traffic

Input: All traffic sliders set to 50.

Result: The system correctly assigns a medium 45-second green light to the N-S cycle, followed by a 45-second green light to the E-W cycle, alternating fairly.

2

Scenario 2: Heavy North-South Traffic

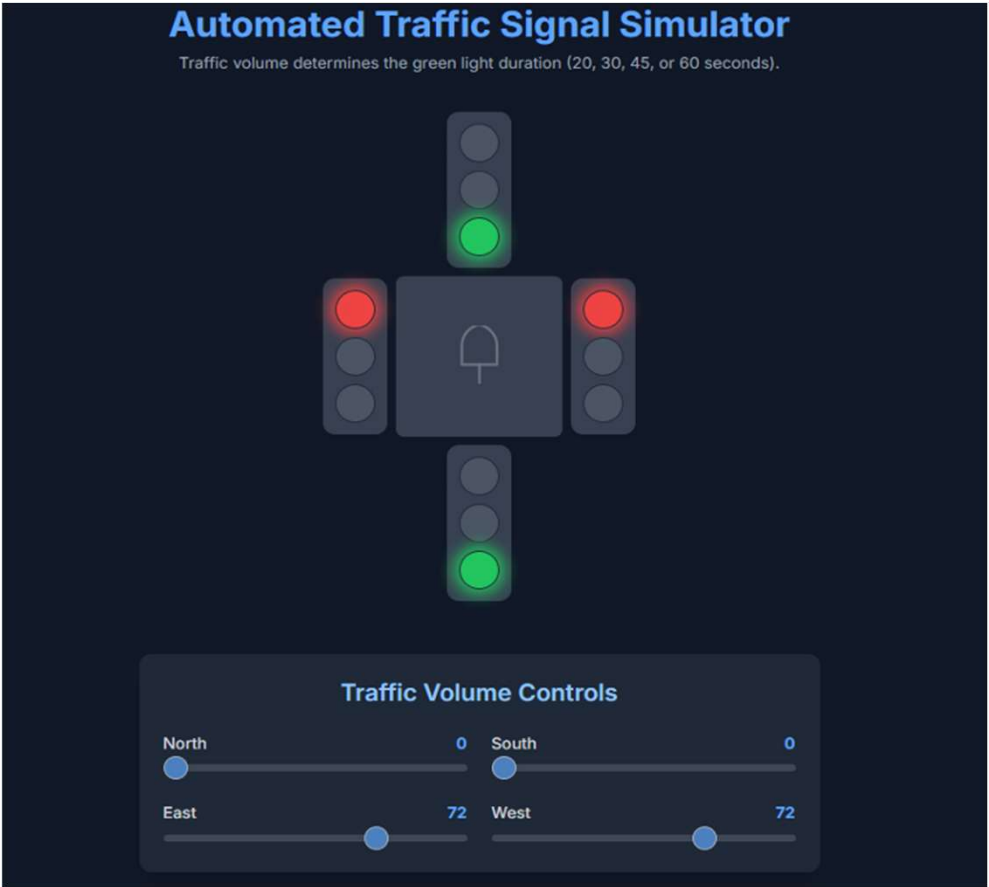
Input: North & South sliders set to 80; East & West set to 20.

Result: The system correctly identifies the high demand and allocates the maximum 60-second green light to the N-S cycle, while giving the minimum 20-second light to the E-W cycle. This visually confirms the adaptive logic is working.

3

Dynamic Flow

The simulation's auto-adjustment feature successfully models traffic flow, preventing sliders from staying static and creating a more realistic environment.



Conclusion

Achievement: This project successfully designed, developed, and validated a proof-of-concept for an adaptive traffic control system through a dynamic web simulation.

Key Takeaway: The simulation clearly proves that a data-driven, adaptive approach is far more efficient at managing traffic flow than traditional fixed-time models.

Limitations

The current model is a single-intersection simulation. It does not account for real-world complexities like pedestrian crossings, emergency vehicles, or the "network effect" of multiple coordinated intersections.

Future Scope

The most critical next step is to replace the manual sliders with a real-time data input. This would involve integrating a Computer Vision AI model (using OpenCV or YOLO) to analyze live CCTV footage, count vehicles, and feed that data into the controller logic, creating a truly autonomous system.

References

- Mozilla Developer Network (MDN) Web Docs. (For HTML, CSS, and JavaScript standards).
- Tailwind CSS. (Official Documentation for the CSS framework).
- Traffic Engineering Handbook, 7th Ed. (For general concepts of traffic flow and signal phasing).
- "A Review of Adaptive Traffic Control Systems." International Journal of Civil Engineering and Technology. (For background on systems like SCATS and SCOOT).
- Redmon, J., & Farhadi, A. (2018). "YOLOv3: An Incremental Improvement." (For future scope research on real-time object detection).